



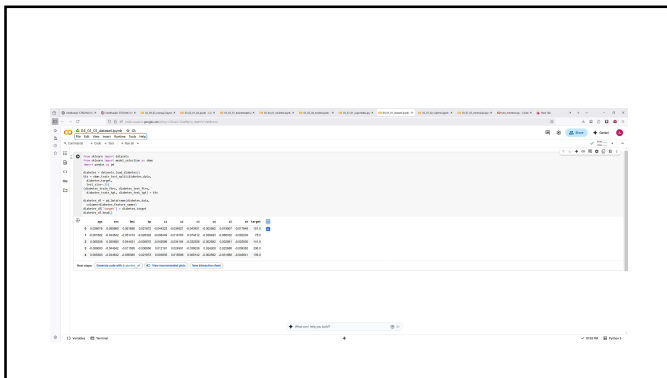
4.1 Jednostavni regresijski skup podataka

Attributes quantitative measure of disease progression

	age	sex	bmi	bp	s1	s2	s3	s4	s5	s6	target
0	0.038076	0.050680	0.061696	0.021872	-0.044223	-0.034821	-0.043401	-0.002592	0.019907	-0.017646	151.0
1	-0.001882	-0.044642	-0.051474	-0.026328	-0.008449	-0.019163	0.074412	-0.039493	-0.068332	-0.092204	75.0
2	0.085299	0.050680	0.044451	-0.005670	-0.045599	-0.034194	-0.032356	-0.002592	0.002861	-0.025930	141.0
3	-0.089063	-0.044642	-0.011595	-0.036656	0.012191	0.024991	-0.036038	0.034309	0.022688	-0.009362	206.0
4	0.005383	-0.044642	-0.036385	0.021872	0.003935	0.015596	0.008142	-0.002592	-0.031988	-0.046641	135.0

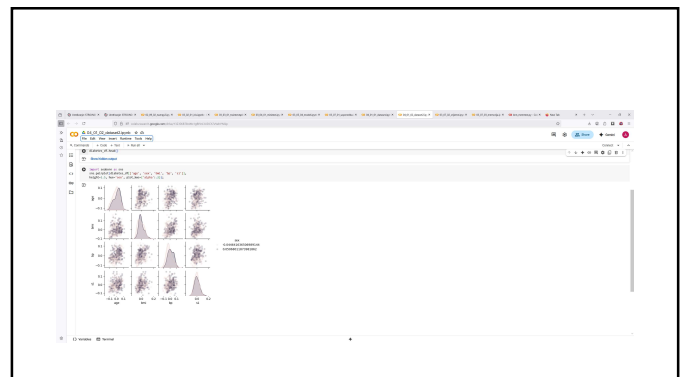
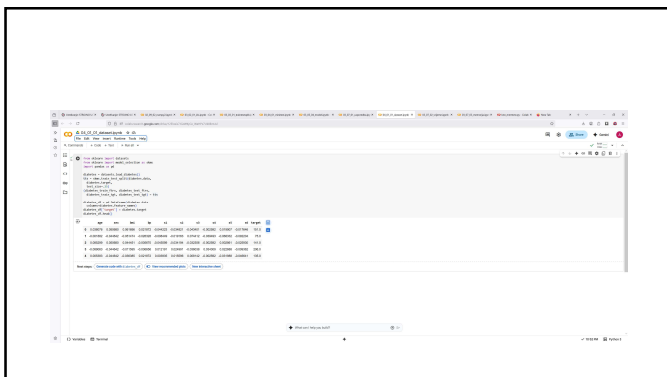
Sample Data * Sklearn Diabetes Dataset contains 442 rows of data

Google
colab



	age	sex	bmi	bp	s1	s2	s3	s4	s5	s6	target
0	0.038076	0.050680	0.061696	0.021872	-0.044223	-0.034821	-0.043401	-0.002592	0.019907	-0.017646	151.0
1	-0.001882	-0.044642	-0.051474	-0.026328	-0.008449	-0.019163	0.074412	-0.039493	-0.068332	-0.092204	75.0
2	0.085299	0.050680	0.044451	-0.005670	-0.045599	-0.034194	-0.032356	-0.002592	0.002861	-0.025930	141.0
3	-0.089063	-0.044642	-0.011595	-0.036656	0.012191	0.024991	-0.036038	0.034309	0.022688	-0.009362	206.0
4	0.005383	-0.044642	-0.036385	0.021872	0.003935	0.015596	0.008142	-0.002592	-0.031988	-0.046641	135.0

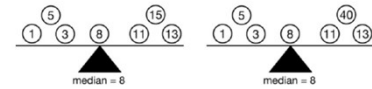
Google
colab



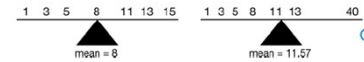
4.2. Regresija najbližih susjeda i sumarna statistika

- k-NN koraci kod klasifikacije:
 - Opišite sličnosti između parova primjera.
 - Odaberite nekoliko najbližnjih primjera.
 - Odabrane primjere spojite u jedan odgovor.

4.2.1. Mjere sredine: medijan i srednja vrijednost



- $\text{sum}(\text{distance}(s, \text{mean}) \text{ for } s \text{ in smaller}) = \text{sum}(\text{distance}(b, \text{mean}) \text{ for } b \text{ in bigger})$
- To vrijedi jedino za $\text{mean} = \text{sum}(d)/\text{len}(d)$



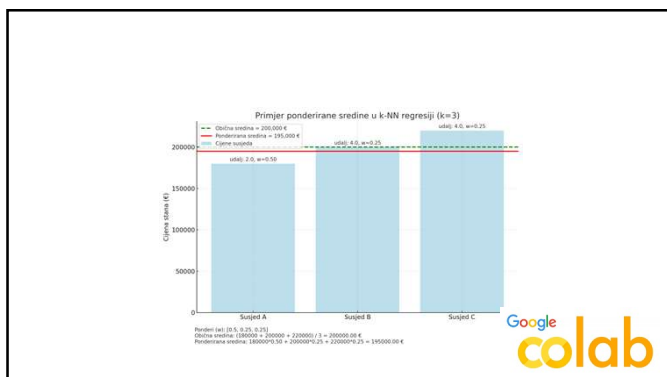
Google
colab

```
import numpy as np
podaci = np.array([1, 3, 5, 8, 11, 13, 15])
print("bez anomalija")
print(np.mean(podaci), np.median(podaci))
podaci_sa_anomalijom = np.array([1, 3, 5, 8, 11, 13, 40])
print("sa anomalijom")
print(np.mean(podaci_sa_anomalijom), np.median(podaci_sa_anomalijom))
```

bez anomalija
8.0 8.0
sa anomalijom
11.57 8.0

Ponderirana srednja vrijednost ?

- Cijene stanova [180000, 200000, 220000]
- Kvadratura stana [2.0, 4.0, 4.0] kao udaljenost značajke od novog testnog primjera !
- Ponder = 1 / kvadratura stana = [0.50, 0.25, 0.25]
- Srednja vrijednost ponderirana = Cijene stanova x Ponder =...
- Sum $((180000, 200000, 220000) \times [0.50, 0.25, 0.25]) = 195000$



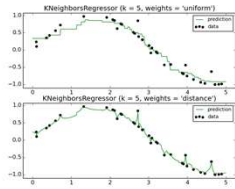
```
[4]: import numpy as np
podaci = np.array([1, 3, 5, 8, 11, 13, 15])
print("bez anomalija")
print(np.mean(podaci), np.median(podaci))
podaci_sa_anomalijom = np.array([1, 3, 5, 8, 11, 13, 40])
print("sa anomalijom")
print(np.mean(podaci_sa_anomalijom), np.median(podaci_sa_anomalijom))

bez anomalija
8.0 8.0
sa anomalijom
11.57 8.0

[4]: udaljenost = np.array([2.0, 4.0, 4.0])
blizina = 1.0 / udaljenost # element-by-element division
tezina = blizina / np.sum(blizina) # normalize sum to one
cijena = np.array([180000, 200000, 220000])
mean = np.mean(cijena)
wgt_mean = np.dot(cijena, tezina)
print("mean", mean)
print("weighted mean", wgt_mean)
```

mean: 195000.0
weighted mean: 195000.0

4.2.2. Izgradnja k-NN regresijskog modela



Google
colab

```
from sklearn import datasets
from sklearn import model_selection as skms
from sklearn import neighbors
from sklearn import metrics

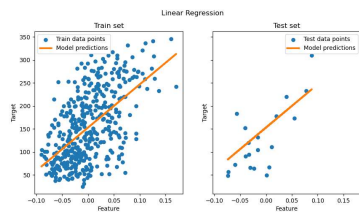
diabetes = datasets.load_diabetes()
diabetes_train_ftrs, diabetes_test_ftrs, diabetes_train_tgt, diabetes_test_tgt = skms.train_test_split(dia

km = neighbors.KNeighborsRegressor(n_neighbors=5)
fit = km.fit(diabetes_train_ftrs, diabetes_train_tgt)
preds = fit.predict(diabetes_test_ftrs)
metrics.mean_squared_error(diabetes_test_tgt, preds)

3389.661728395862

[ ] Start coding or generate with AI.
```

4.3. Linearna regresija i pogreške



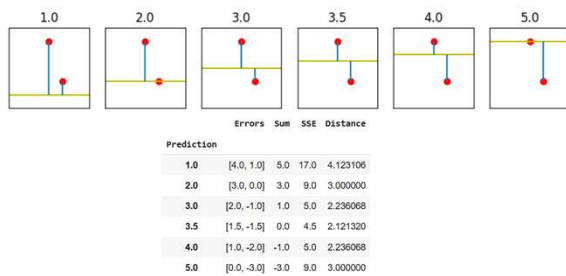
Google
colab

```
import numpy as np
import matplotlib.pyplot as plt

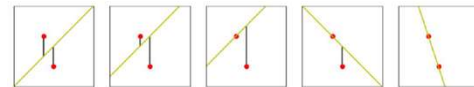
def axis_helper(ax, lims):
    'clean up axes'
    ax.set_xlim(lims)
    ax.set_xticks([])
    ax.set_ylim(lims)
    ax.set_yticks([])
    ax.set_aspect('equal')

# tražimo udaljenost između dvije točke
d = np.array([1, 5], [4, 2])
# x je prvi stupac, a y drugi stupac
x, y = d[:, 0], d[:, 1]

vodoravni_gravci = np.array([1, 2, 3, 3.5, 4, 5])
rezultati = []
fig, axes = plt.subplots(1, 6, figsize=(16, 5))
for h, line, ax in zip(vodoravni_gravci, axes.flat):
    axis_helper(ax, (0, 6))
    ax.set_title(h)
    # crtanje
    ax.plot(x, y, 'ro')
```



4.3.2. Nagib pravca



- $\text{distance}(\text{prediction}, y) = \text{distance}(ax + b, y)$
- $\text{sum}(\text{distance}(ax + b, y) \text{ for } x, y \text{ in } D)$

$$\sum_{(x,y) \in D} ((ax + b) - y)^2$$

Google
colab

```

import numpy as np
import matplotlib.pyplot as plt
import pandas as pd

def axis_helper(ax, lims):
    ax.set_xlim(lims)
    ax.set_yticks([])
    ax.set_xlabel('')
    ax.set_ylabel('')
    ax.set_aspect('equal')

def process(D, model, ax):
    x, y = D[:, 0], D[:, 1]
    a, b = model
    axis_helper(ax, (0, 8))
    ax.plot(x, y, 'ro')
    # crtaje pravcu
    helper_x = np.linspace(0, 8)
    helper_line = a * helper_x + b
    ax.plot(helper_x, helper_line, color='y')
    # crtaje pravcu
    prediction = model.predict(x)
    ax.plot(x, prediction, color='b')
    # crtaje pravcu

```

4.3 Izvođenje linearne regresije

- $\text{rdot}(\text{weights_wo}, \text{features}) + \text{wgt_b}$ (ako weights_wo ne uključuje vrijednost b)
- $\text{rdot}(\text{weights}, \text{features_p1})$
- $\text{distance}(\text{prediction}, \text{actual})$

$$y_{\text{pred}} = \sum_{\text{ftrs}} w_f x_f = w \cdot f$$

Google
colab

```

from sklearn import datasets
from sklearn import model_selection as skms
from sklearn import metrics
from sklearn import linear_model

diabetes = datasets.load_diabetes()
diabetes_train_ftrs, diabetes_test_ftrs, diabetes_test_tgt = skms.train_test_split(dia

lr = linear_model.LinearRegression()
fit = lr.fit(diabetes_train_ftrs, diabetes_train_tgt)
preds = fit.predict(diabetes_test_ftrs)
metrics.mean_squared_error(diabetes_test_tgt, preds)

```

4.4 Optimizacija: odabir najboljeg odgovora

1. Slučajno pogađanje
2. Slučajan korak
3. Pametan korak
4. Proračunati prećak

Google
colab

4.4.1. Slučajno pogađanje

```

import numpy as np
tgt = np.array([3, 5, 8, 10, 12, 15])
num_guesses = 10
results = []
for g in range(num_guesses):
    guess = np.random.uniform(low=tgt.min(), high=tgt.max())
    print(guess)
    total_dist = np.sum((tgt - guess)**2)
    print(tgt)
    print(total_dist)
    results.append((total_dist, guess))
best_guess = sorted(results)[0][1]
print("
print(best_guess)

```



4.4.2. Slučajan korak



Google
colab

4.4.2. Slučajan korak

```
import numpy as np
tgt = np.array([3, 5, 8, 10, 12, 15])
num_steps = 100
step_size = .05
best_guess = np.random.uniform(low=tgt.min(), high=tgt.max())
best_dist = np.sum((tgt - best_guess)**2)
print(best_guess)
print(best_dist)
print("_____")
for s in range(num_steps):
    new_guess = best_guess + (np.random.choice([-1, 1]) * step_size)
    new_dist = np.sum((tgt - new_guess)**2)
    print(new_guess)
    print(new_dist)
    if new_dist < best_dist:
        best_guess, best_dist = new_guess, new_dist
print("_____")
print(best_guess)
```

4.4.3. Pametan korak



Google
colab

4.4.3. Pametan korak

```
import numpy as np
tgt = np.array([3, 5, 8, 10, 12, 15])

num_steps = 1000
step_size = .02
best_guess = np.random.uniform(low=tgt.min(), high=tgt.max())
best_dist = np.sum((tgt - best_guess)**2)
print("start:", best_guess)
for s in range(num_steps):
    # np.newaxis is needed to align the minus
    guesses = best_guess + (np.array([-1, 1]) * step_size)
    dists = np.sum((tgt[:, np.newaxis] - guesses)**2, axis=0)
    better_idx = np.argmin(dists)
    if dists[better_idx] < best_dist:
        break
    best_guess = guesses[better_idx]
    best_dist = dists[better_idx]
print("end:", best_guess)
```

start: 3.2182752641091463
end: 8.838275264109045

4.4.4. Proračunati prečac



Google
colab

4.4.4. Proračunati prečac

```
import numpy as np
tgt = np.array([3, 5, 8, 10, 12, 15])
print("mean:", np.mean(tgt))
```

mean: 8.833333333333334

4.4.5 Primjena na linearnu regresiju

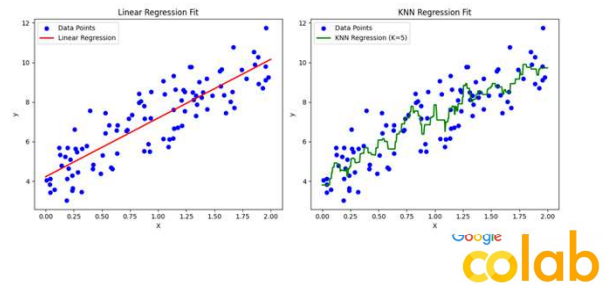
1 ? 2
3 4

4.5 Jednostavna evaluacija i usporedba regresora

Korijen srednje kvadratne pogreške

$$RMSE = \sqrt{\sum_{i=1}^n \frac{(\hat{y}_i - y_i)^2}{n}}$$

4.5.2 Učinkovitost učenja



4.5.2 Učinkovitost učenja

```
import numpy as np
from sklearn import (datasets, neighbors,
                     model_selection as skms,
                     linear_model, metrics)
diabetes = datasets.load_diabetes()
tts = skms.train_test_split(diabetes.data,
                           diabetes.target,
                           test_size=0.25)
(diabetes_train, diabetes_test,
 diabetes_train_tgt, diabetes_test_tgt) = tts
models = ('knn': neighbors.KNeighborsRegressor(n_neighbors=5),
         'linreg': linear_model.LinearRegression())
for name, model in models.items():
    fit = model.fit(diabetes_train, diabetes_train_tgt)
    preds = fit.predict(diabetes_test)
    score = np.sqrt(metrics.mean_squared_error(diabetes_test_tgt, preds))
    print("{}: {:.2f}".format(name, score))
```

knn : 63.64
linreg : 51.18

4.5.3 Korištenje resursa u regresiji

- perf2a.py dostupan na moodle.srce.hr pod temom 4.
- Pokreće se iz naredbenog retka sa argumentima
- Možemo koristiti COLAB terminal
- Potrebno: pip install memory_profiler
- Pogledajte kod!

